Exercise 1 — Bubble Sort / Selection Sort (General)

Requirements

- A static int array, sorted in ascending order two separate ways: once with bubble sort, once with selection sort. Unlike most of the exercises below, both algorithms are required here, not a choice between them.
- Implement each as its own method, run each on its own copy of the original array (so sorting with the first algorithm doesn't leave the array already-sorted before the second one runs), and print both results — they should be identical, since the point is comparing two ways of reaching the same correct answer, not comparing two different answers.

Exercise 2 — Selection Sort / Bubble Sort

Requirements

- Pick one algorithm (either is fine) — this exercise only asks for a single sort, not both.
- Input: `[34, 12, 25, 9, 78, 31]`
- Expected sorted output: `[9, 12, 25, 31, 34, 78]`

Exercise 3 — Selection Sort / Bubble Sort with Step Tracking

Requirements

- Pick one algorithm (either is fine).
- Input: `[45, 11, 32, 5, 27, 19]` → sorted: `[5, 11, 19, 27, 32, 45]`
- Count and print the number of comparisons and swaps performed — this is required, not an optional extra.

Expected counts for this input

- Bubble sort (standard version, no early-exit optimization): 15 comparisons, 10 swaps.
- Selection sort: 15 comparisons, 4 swaps.
- Both algorithms happen to make 15 comparisons on any 6-element array — that's not specific to this data, it's just how many comparisons a non-early-exit pass structure always makes for n=6 (n(n–1)/2). The swap counts, on the other hand, genuinely depend on how out-of-order the input is, which is why they differ between the two algorithms here. If you implement bubble sort with an early-exit optimization (stopping once a full pass makes zero swaps), its comparison count could come out lower than 15 — that's expected, not a mismatch with the number above.

Exercise 4 — Selection Sort / Bubble Sort

Requirements

- Pick one algorithm (either is fine).
- Input: `[29, 10, 14, 37, 13]`
- Expected sorted output: `[10, 13, 14, 29, 37]`

Exercise 5 — Selection Sort (with Intermediate Steps)

Requirements

- Selection sort specifically (not a choice this time), printing three things: the original array, the array's state after every pass, and the final sorted array.
- The original doesn't give a specific input array for this one — any array works; here's the expected step-by-step shape using `[29, 10, 14, 37, 13]` as an example:

```
Original: [29, 10, 14, 37, 13]
After pass 1: [10, 29, 14, 37, 13]
After pass 2: [10, 13, 14, 37, 29]
After pass 3: [10, 13, 14, 37, 29]
After pass 4: [10, 13, 14, 29, 37]
After pass 5: [10, 13, 14, 29, 37]
```

Note on the trace

- Passes 3 and 5 don't visibly change anything here — that's expected: selection sort always runs one pass per element regardless of whether that pass happens to find anything already in the right place. Print every pass, even the ones that don't change the array, so the step count matches what selection sort actually does.

Exercise 6 — Insertion Sort / Merge Sort

Requirements

- Pick one algorithm (either is fine).
- Input: `[45, 11, 32, 5, 27, 19]` (the same six values used in an earlier exercise, different algorithm this time) → sorted: `[5, 11, 19, 27, 32, 45]`
- Count and print comparisons and swaps — required, not an optional extra.

Counting for insertion sort

- Insertion sort doesn't really do "swaps" in the bubble/selection sense — instead it shifts already-placed elements one slot over to make room for the one being inserted. Count comparisons and shifts (element moves) rather than trying to force a swap count onto it. For this input: 13 comparisons, 10 shifts.

Counting for merge sort

- Merge sort has no swaps at all — it builds a new merged array by copying elements in, it never exchanges two elements in place. If you pick merge sort, count comparisons made during the merge step, and simply note that a "swap count" doesn't apply to this algorithm rather than reporting 0 as if that meant something.

Exercise 7 — Linear Search

Requirements

- Array: `[45, 23, 67, 12, 34, 89, 50]` — deliberately unsorted, which is fine for linear search; unlike binary search, it doesn't need the data in any particular order.
- Prompt for a number from the keyboard; scan the array from the start, returning the index of the first match, or -1 if the number never appears.

Exercise 8 — Binary Search

Requirements

- Sorted array: `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]`
- Prompt for a number from the keyboard. Implement the search algorithm yourself — no `Arrays.binarySearch()` or similar built-in.

Algorithm

- Keep a low and high index spanning the whole array. Repeatedly compute the midpoint; if the value there matches, return that index. If the target is smaller, narrow the range to the left half (high = mid – 1); if larger, narrow to the right half (low = mid + 1). If the range becomes empty (low > high) without a match, return -1.

Exercise 9 — Binary Search

Requirements

- Sorted array: `[3, 6, 9, 12, 15, 18, 21, 24]`
- Same algorithm as before — low/high indices, compare against the midpoint, narrow the range by half each step — applied to different data. Prompt for a number and return its index, or -1 if it isn't present.

Exercise 10 — Longest Palindromic Substring

Requirements

- A palindrome here means length ≥ 2 — every single character trivially reads the same forwards and backwards, but that doesn't count as a found result. Print "No palindromic substring found." only if nothing of length 2 or more qualifies.
- A workable approach: for every possible center point in the string (each single character, for odd-length palindromes, and each gap between two adjacent characters, for even-length ones), expand outward in both directions while the characters on each side keep matching, and track the longest match found across all centers. This runs in O(n×n) time, which is more than fast enough for typical exercise-sized input.

Example

- Input: `abacdfgdcabba`
- Output: `abba`
- (There's a shorter 3-letter palindrome, "aba", earlier in the string, and a 2-letter "bb" inside "abba" itself — but "abba" at length 4 is the single longest match, with no other 4-letter-or-longer palindrome anywhere else in the string.)

Exercise 11 — Sorting Student Exam Scores (Bubble Sort)

Requirements

- Read N student scores (N ≤ 100) from the keyboard, sort them with bubble sort in descending order, then display the sorted list, the highest and lowest scores, and the total number of swaps the sort performed.

Sample input

- 5 students: 7.5, 8.0, 6.5, 9.0, 7.0

Expected output

- Sorted (descending): `9.0 8.0 7.5 7.0 6.5`
- Highest score: `9.0`, lowest score: `6.5`
- Total number of swaps: `5` — tracing a standard bubble sort by hand on this exact input gives 3 swaps on the first pass, 1 on the second, 1 on the third, and 0 on the remaining two passes, for a total of 5. (The original assignment states 6 for this sample; that doesn't match any standard bubble sort implementation run against these numbers — 5 is correct here.)

Exercise 12 — Warehouse Product Search (Linear Search + Binary Search)

Requirements

- An array of product codes, already sorted ascending. Search for a code two ways — linear search and binary search — printing the position found and the elapsed time for each (e.g. via `System.currentTimeMillis()` around each search call).

Sample input/output

- Codes: `[101, 203, 305, 407, 512, 678, 789]`, searching for `512`: both searches correctly find it at position `4` (0-indexed).

```
Linear Search:
Found at position 4
Time: 0 ms

Binary Search:
Found at position 4
Time: 0 ms
```

A note on the timing

- With an array this small, `0 ms` for both is expected and doesn't mean the two algorithms performed equally — millisecond-resolution timing can't distinguish operations that both complete in a handful of microseconds. The point of this exercise is wiring up the timing code correctly, not observing a real gap; if you want to actually see binary search pull ahead, try it against a generated array of a few hundred thousand elements, and consider `System.nanoTime()` instead of currentTimeMillis() for finer resolution.

Exercise 13 — Ranking Students & Searching (Selection Sort + Binary Search)

Requirements

- Read N students' names and scores. Sort with selection sort by score descending; when two students tie on score, break the tie by name, ascending.
- Then allow searching for a student by name.

Sample input

- 4 students: Lan–8.5, Minh–9.0, Hoa–8.5, An–7.0

Expected ranking

- Minh (9.0) is highest, unambiguously. Lan and Hoa are tied at 8.5, so the name tiebreak decides between them — and alphabetically, "Hoa" comes before "Lan". The correct order is:

```
1. Minh - 9.0
2. Hoa - 8.5
3. Lan - 8.5
4. An - 7.0
```

(The original assignment's sample output lists Lan above Hoa at ranks 2 and 3 — that's inconsistent with its own stated tiebreak rule, since 'H' sorts before 'L'. The ordering above is what the stated rule actually produces. Searching for "Lan" afterward correctly reports rank 3, not rank 2 as the original's sample claims.)

Why the binary search needs adapting

- Binary search fundamentally requires the data to be sorted by the same value you're searching on — but here the list is sorted by score, and the search is by name, which has no relationship to score order at all. A standard binary search by name run directly against this score-sorted list simply won't work; there's no ordering for it to narrow down against.
- The fix: keep a second copy of the student list sorted by name specifically for searching. Binary search that copy by name to find the student, then look up (or store alongside each entry) their position in the score-sorted ranking to report their rank. This is what "adapt the search logic" means here — binary search still gets used, just against data that's actually sorted the right way for it.